

CampusEats — External SOAP Partner Integration

CS543 Web Services — Assignment 3

Team ID: [04]

Team Members

1. Akshay Kumar-20251651012
2. Sundram Tripathi-20251651091
3. Abhinav Kumar Chandravanshi -20251651004
4. Ankit Kumar -20251651022

External Partner

Payment Gateway

Integrated Operation

charge

Submission

This document describes the design-time integration of an external SOAP Payment Gateway with CampusEats, including the partner WSDL, SOAP messages, HTTP binding, service discovery, and fault mapping.

1. Context

CampusEats will integrate an external Payment Gateway as its SOAP partner. The SOAP operation selected for integration is charge, which is used to process the payment for a CampusEats order. SOAP is appropriate for this payment edge because it provides a strong formal contract, standardized message and fault handling, and support for message-level security and transactional processing. The remaining CampusEats services can continue using REST because their internal communication is lightweight and resource-oriented.

2. HTTP Binding

CampusEats sends the charge SOAP request to the external Payment Gateway using HTTP POST. The endpoint and SOAPAction are taken from the partner WSDL. The SOAP envelope containing the credentials and charge operation is carried as the HTTP request body.

Endpoint:

`https://api.paymentpartner.example.com/soap/payment`

HTTP Request:

`POST /soap/payment HTTP/1.1`

`Host: api.paymentpartner.example.com`

Content-Type: text/xml; charset=utf-8

SOAPAction: "http://campuseats.example.com/payment/charge"

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<soap:Envelope
```

```
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
```

```
  xmlns:pay="http://campuseats.example.com/payment">
```

```
  <soap:Header>
```

```
    <pay:Credentials>
```

```
      <pay:merchantId>CE_MERCHANT_001</pay:merchantId>
```

```
      <pay:apiKey>DEMO_API_KEY_123</pay:apiKey>
```

```
    </pay:Credentials>
```

```
  </soap:Header>
```

```
  <soap:Body>
```

```
    <pay:charge>
```

```
      <pay:orderId>ORD1001</pay:orderId>
```

```
      <pay:amount>499.00</pay:amount>
```

```
      <pay:currency>INR</pay:currency>
```

```
      <pay:paymentToken>tok_demo_123</pay:paymentToken>
```

```
    </pay:charge>
```

```
  </soap:Body>
```

```
</soap:Envelope>
```

3. Discovery

CampusEats would discover the external Payment Gateway through a modern service catalogue/registry. The catalogue contains the business name, service name, endpoint, and a pointer to the WSDL contract. CampusEats can use the WSDL pointer to obtain the partner contract before binding its SOAP consumer to the charge operation.

Registry Entry

Field	Value
Business	CampusEats Payment Partner
Service	PaymentGatewayService
Endpoint	https://api.paymentpartner.example.com/soap/payment
WSDL / tModel-style pointer	https://api.paymentpartner.example.com/wsdl/payment.wsdl

The WSDL pointer identifies where the partner's SOAP contract can be obtained, while the endpoint identifies where the SOAP request is sent. No live UDDI server is required for this design.

4. Fault Mapping

The external Payment Gateway may return the partner-specific fault `card_declined` when a payment cannot be completed.

CampusEats will not expose this partner-specific vocabulary to its students or callers. The Order Service translates the partner fault into the error defined by the CampusEats `placeOrder` contract.

Mapping

External Payment Gateway CampusEats

<code>card_declined</code>	<code>payment declined</code>
----------------------------	-------------------------------

Flow

Payment Gateway

```

|
| card_declined
v

```

Order Service

```

|
| translate
v

```

CampusEats `placeOrder`

```

|
| payment declined
v

```

Caller / Student

This mapping keeps the external partner's implementation and error vocabulary hidden behind the CampusEats service contract.

